

SISTEMAS LEGADOS 26/27

Memoria de la Práctica 1



MIERCOLES B 15:00-18:00

GRUPO 03

Alejo Beamonte Andreu (899577)

Izan Celorrio Caballero (900013)

Pablo Redondo Buenacasa (899595)

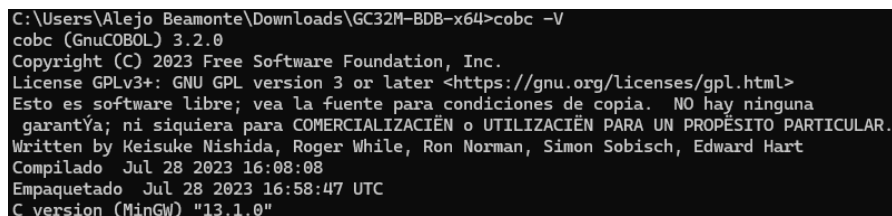
Índice

1. Objetivo 1.....	3
1.1. Compilación y adaptación del código COBOL.....	3
Figura 1 - Características del compilador utilizado.....	3
1.2. Correcciones del código para la ejecución del sistema recuperado.....	4
Figura 2 - Descubrimiento de la ausencia de movimientos.ubd.....	4
1.3. Prueba de las funcionalidades del menú.....	5
2. Objetivo 2.....	6
2.1. Modificación de “Ordenar transferencia”.....	6
2.2. Implementación de “Listado de transferencias”.....	7
3. Objetivo 3.....	8
3.1. Objetivo 3a.....	8
3.2. Objetivo 3b.....	8
3.3. Objetivo 3c.....	9
3.4. Objetivo 3d.....	10
4. Empaquetado, distribución y control de versiones.....	10
4.1. Distribución standalone en Windows y dependencias de tiempo de ejecución.....	10
4.2. Repositorio de control de versiones (Git y GitHub).....	11
5. Reparto del trabajo.....	11

1. Objetivo 1

1.1. Compilación y adaptación del código COBOL

Para compilar el código recuperado se utilizó **GnuCOBOL 3.2 (28Jul2023) MSYS2 64-bit**. Una vez configurado el entorno mediante `set_env.cmd`, se comprobó el correcto funcionamiento del compilador ejecutando `cobc -V`.



```
C:\Users\Alejo Beamonte\Downloads\GC32M-BDB-x64>cobc -V
cobc (GnuCOBOL) 3.2.0
Copyright (C) 2023 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <https://gnu.org/licenses/gpl.html>
Esto es software libre; vea la fuente para condiciones de copia. NO hay ninguna
garantía; ni siquiera para COMERCIALIZACIÓN o UTILIZACIÓN PARA UN PROPÓSITO PARTICULAR.
Written by Keisuke Nishida, Roger While, Ron Norman, Simon Sobisch, Edward Hart
Compilado Jul 28 2023 16:08:08
Empaquetado Jul 28 2023 16:58:47 UTC
C version (MinGW) "13.1.0"
```

Figura 1 - Características del compilador utilizado

El primer intento de compilación del programa principal se realizó directamente sobre el código original mediante `cobc -x BANK1.cbl`, produciéndose numerosos errores de sintaxis. En primer lugar, se detectó que algunas líneas eran interpretadas utilizando el formato fijo tradicional de COBOL, lo que provocaba problemas en líneas largas. Para evitarlo se pasó a utilizar el formato libre mediante la opción `-free`.

También se detectó una incompatibilidad en las instrucciones de posicionamiento de pantalla. El código original utilizaba expresiones como `DISPLAY (8 15) "Numero de tarjeta:"`, mientras que GnuCOBOL requería separar las coordenadas mediante una coma: `DISPLAY (8, 15) "Numero de tarjeta:"`. Se realizaron las mismas adaptaciones en las instrucciones `ACCEPT` y, cuando se combinaba el posicionamiento con `ON EXCEPTION`, fue necesario utilizar la forma `AT`, por ejemplo `ACCEPT CHOICE AT 2480 ON EXCEPTION`.

Durante la adaptación también aparecieron algunos problemas específicos. En `BANK1.cbl` se corrigió una instrucción `DISPLAY` que incluía por error las coordenadas dentro de la propia cadena de texto. En `BANK6.cbl` se encontró una incompatibilidad en un campo de la `SCREEN SECTION` que combinaba `SIGN IS LEADING SEPARATE` con `BLANK ZERO`; se eliminó la cláusula de signo del campo de entrada, manteniendo con signo la variable interna utilizada posteriormente en las operaciones.

Finalmente se compilaron todos los módulos de forma individual. El programa principal se generó mediante `cobc -x -free -std=mf BANK1.cbl` y los módulos auxiliares `BANK2.cbl` a `BANK7.cbl` mediante `cobc -m -free -std=mf BANKn.cbl`. Todos quedaron compilados correctamente. En `BANK4.cbl` y `BANK5.cbl` se obtuvieron únicamente avisos relacionados con saltos `GO TO` entre distintas `SECTION`, que se mantuvieron al no impedir la compilación ni constituir por sí mismos un error funcional.

Finalmente se obtuvo `BANK1.exe`, permitiendo iniciar la aplicación bancaria en Windows. Con ello se cumplió la primera parte del objetivo, consistente en recuperar y compilar el código disponible con un compilador actual.

1.2. Correcciones del código para la ejecución del sistema recuperado

Aunque la aplicación ya compilaba y podía iniciarse, las primeras pruebas mostraron que varias opciones del menú terminaban directamente en la pantalla de error interno. Para localizar el problema se analizaron los valores de los distintos FILE STATUS asociados a los ficheros utilizados por la aplicación.

En varios módulos aparecían comprobaciones del tipo IF FSM <> 30 GO TO PSYS-ERR., pese a que una operación de fichero correcta devolvía el estado 00. Estas condiciones se corrigieron por IF FSM NOT = 00 GO TO PSYS-ERR., aplicándose el mismo cambio a otros estados de fichero como FST y FSE.

Durante el diagnóstico de las opciones que trabajaban con movimientos se añadió temporalmente una visualización del estado del fichero, obteniéndose FILE STATUS: 35.



Figura 2 - Descubrimiento de la ausencia de movimientos.ubd

Este valor permitió localizar la ausencia de movimientos.ubd, que no había sido incluido entre los materiales recuperados. Al estar declarado como ORGANIZATION IS INDEXED, no era suficiente con crear un fichero vacío convencional, por lo que se desarrolló el programa auxiliar CREAR-MOVIMIENTOS.cb1, encargado de generarlo correctamente mediante OPEN OUTPUT. Tras su ejecución se obtuvo FILE STATUS: 00 y el mensaje Fichero movimientos.ubd creado..

Otra incidencia apareció en la opción de compra de entradas. El fichero espectaculos.ubd sí había sido recuperado, pero sus registros correspondían a espectáculos antiguos y el propio programa descartaba aquellos cuya fecha ya había pasado. Para poder comprobar funcionalmente esta opción se creó ACTUALIZAR-ESPECTACULO.cb1, que permitió modificar temporalmente la fecha de uno de los registros a una fecha futura.

Finalmente, el menú principal incluía la opción “7 - Cambiar clave”, pero no se había proporcionado el correspondiente BANK8.cb1, por lo que al seleccionarla se producía el error libcob: error: module 'BANK8' not found. Por este motivo se reconstruyó el módulo utilizando como referencia el comportamiento descrito en el guion de la práctica: introducción de la clave actual y de la nueva clave, actualización si la comprobación era correcta, reducción del número de intentos en caso de error y bloqueo de la tarjeta tras tres intentos fallidos.

Una vez aplicadas estas correcciones, todas las opciones del menú podían abrirse y ser probadas funcionalmente.

1.3. Prueba de las funcionalidades del menú

Una vez conseguido que la aplicación compilara y ejecutara correctamente, se realizaron pruebas sobre todas las funcionalidades disponibles en el menú principal, tal y como exige el enunciado.

Opción	Prueba realizada	Resultado
1 - Consultar saldo	Consulta inicial y tras realizar operaciones	Correcto. El saldo inicial era 0 EUR y posteriormente se actualizó conforme a los movimientos realizados.
2 - Consultar movimientos	Consulta sin movimientos y tras realizar ingresos, retiradas y transferencias	Correcto. Los movimientos quedaron registrados y podían consultarse mediante el filtro de fechas e importe.
3 - Retirar efectivo	Retirada válida y retirada superior al saldo disponible	Correcto. La retirada válida actualizó el saldo y una cantidad superior al saldo disponible fue rechazada.
4 - Ingresar efectivo	Ingreso de 100 EUR	Correcto. El saldo se actualizó y el movimiento apareció posteriormente en la lista de movimientos.
5 - Ordenar transferencia	Transferencia a una cuenta existente y a una cuenta inexistente	Correcto. La transferencia válida modificó los saldos correspondientes y la cuenta inexistente fue rechazada mediante un mensaje de error.
6 - Comprar entradas de espectáculos	Compra de entradas para el espectáculo cuya fecha se actualizó para las pruebas	Correcto. La compra se realizó y el saldo quedó actualizado.
7 - Cambiar clave	Cambio válido, intento con clave incorrecta y agotamiento de los tres intentos	Correcto. La nueva clave quedó guardada y, posteriormente, tras tres errores, la tarjeta quedó bloqueada. Se verificó además que una tarjeta bloqueada no podía volver a iniciar sesión en el sistema.

2. Objetivo 2

2.1. Modificación de “Ordenar transferencia”

El módulo original BANK6.cbl únicamente permitía transferencias inmediatas en el día actual con cargo directo sobre movimientos.ubd. Para cumplir con el Objetivo 2, se rediseñó el programa permitiendo transferencias puntuales en una fecha dada (DD-MM-AAAA) o periódicas mensuales en un día fijo de cada mes (01-31).

Para evitar adulterar el libro contable de movimientos.ubd (que exige registrar el saldo resultante tras cada operación y tiene longitud de registro fija), se creó un nuevo fichero indexado: transferencias.ubd (RECORD KEY IS TRF-NUM), inicializado mediante el programa auxiliar CREAR-TRANSFERENCIAS.cbl. En este fichero se almacena el número de orden, cuentas origen y destino, titular, importe, tipo (P o M), fecha puntual o día del mes, y su estado (E para ejecutada o P para pendiente).

En la interfaz de BANK6.cbl se reorganizó la pantalla para mostrar todos los datos de forma simultánea y bien alineada (etiquetas en columna 15 e inputs en columna 44), corrigiendo un solapamiento en el campo de modalidad (Perio_ica) y precargando automáticamente la fecha actual (CURRENT-DATE).

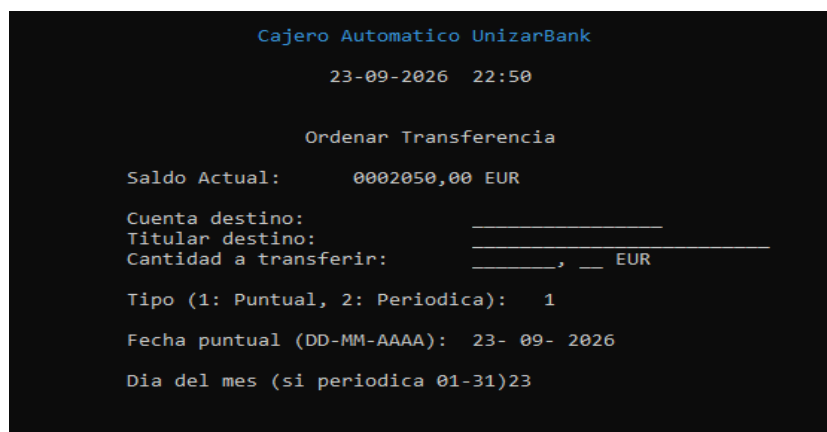


Figura 3 - Pantalla de "Ordenar Transferencia" con selección de modalidad y fecha

El procesamiento implementa validaciones estrictas: existencia de la cuenta destino en tarjetas.ubd, bloqueo de transferencias a la propia tarjeta, importe positivo y fecha no anterior a la actual. Si la transferencia puntual es para el día en curso y existe saldo suficiente, se anotan los movimientos contables en movimientos.ubd y se guarda en transferencias.ubd como ejecutada (E). Si la fecha es futura o la orden es periódica, se registra como pendiente (P) sin descontar saldo anticipadamente. Cabe señalar que, al tratarse del software de un cajero automático (ATM), la aplicación se limita a registrar estas órdenes en transferencias.ubd; la ejecución y cargo automático al cumplirse las fechas futuras corresponde a los procesos batch del sistema central del banco.

Durante las pruebas se detectó un fallo intermitente al pulsar ESC y reabrir la opción: el fichero movimientos.ubd quedaba bloqueado por Berkeley DB debido a una doble apertura con OPEN I-O heredada del código original. El problema se resolvió unificando la consulta de saldo en una única lectura en modo OPEN INPUT con cierre inmediato, y declarando PROGRAM-ID. BANK6 IS INITIAL PROGRAM. para garantizar la liberación limpia de descriptores en cada invocación.

2.2. Implementación de “Listado de transferencias”

Para dar cumplimiento a la segunda parte del Objetivo 2, se desarrolló el nuevo módulo **BANK9.cbl** y se reorganizó el menú principal en **BANK1.cbl** para alojar 8 opciones, situando el nuevo listado como opción 6 y desplazando las opciones previas de comprar entradas y cambio de clave a las opciones 7 y 8 respectivamente.

El funcionamiento de **BANK9.cbl** se divide en dos fases:

1. Pantalla de filtrado: Solicita al usuario un intervalo de búsqueda mediante **Fecha desde** y **Fecha hasta (DD-MM-AAAA)**, permitiendo pulsar Enter directamente para consultar todo el historial. Se valida que ambas fechas sean correctas y que la fecha de inicio no exceda a la de fin.
2. Procesamiento y visualización: El programa recorre secuencialmente **transferencias.ubd**, filtrando las órdenes pertenecientes a la tarjeta en sesión (**TRF-CTA-ORIGEN = TNUM**) que se encuentren dentro del rango temporal seleccionado.

Los resultados se presentan en una tabla paginada (hasta 10 registros por pantalla) que detalla: fecha o día de cargo, modalidad (**PUNTUAL** o **PERIODICA**), cuenta de destino, importe y estado de la operación.

Cajero Automatico UnizarBank					
23-09-2026 23:08					
LISTADO DE TRANSFERENCIAS					
FECHA/DIA	TIPO	CUENTA DESTINO	IMPORTE (EUR)	ESTADO	
DIA 23	PERIODICA	0000000000000002	0000050,00 EUR	EJECUTADA	
DIA 23	PERIODICA	0000000000000002	0000050,00 EUR	EJECUTADA	
DIA 23	PERIODICA	0000000000000002	0000500,00 EUR	EJECUTADA	
23-09-2026	PUNTUAL	0000000000000002	0000020,00 EUR	EJECUTADA	
DIA 23	PERIODICA	0000000000000002	0000040,00 EUR	EJECUTADA	
DIA 30	PERIODICA	0000000000000002	0001000,00 EUR	PENDIENTE	

Figura 4 - Pantalla de "Listado de Transferencias" mostrando la tabla paginada con operaciones puntuales y periódicas y sus estados

La determinación del estado de cada transferencia se implementó con una lógica adaptada a la naturaleza de cada operación:

- En transferencias puntuales: Si la fecha es menor o igual a la del sistema (**CURRENT-DATE**), se marca en verde como **EJECUTADA**; si la fecha es futura, figura en amarillo como **PENDIENTE**.
- En transferencias periódicas: Se evalúa el día mensual de cargo frente al día actual. Si el día ya ha sido alcanzado o superado en el mes en curso (**TRF-DIA-MES <= DIA**), se muestra como **EJECUTADA** (indicando que en el ciclo mensual actual ya se ha cumplido); si el día aún no ha llegado, figura como **PENDIENTE**.

Al igual que en **BANK6.cbl**, el fichero se abre en modo solo lectura (**OPEN INPUT**) y se cierra inmediatamente tras la carga en memoria de los registros, evitando bloqueos. Asimismo, se incorporó la directiva **PROGRAM-ID. BANK9 IS INITIAL PROGRAM.** para garantizar la inicialización limpia del módulo en sucesivas consultas.

3. Objetivo 3

3.1. Objetivo 3a

Se modificó la opción “4 - Ingresar efectivo” para que el usuario no introduzca directamente una cantidad, sino el número de billetes de **10, 20 y 50 euros** que desea ingresar. Para ello se añadieron tres campos de entrada en BANK5.cb1 y se calculó automáticamente el importe total mediante $\text{COMPUTE EURENT-USUARIO} = (\text{BILLETES-10} * 10) + (\text{BILLETES-20} * 20) + (\text{BILLETES-50} * 50)$.

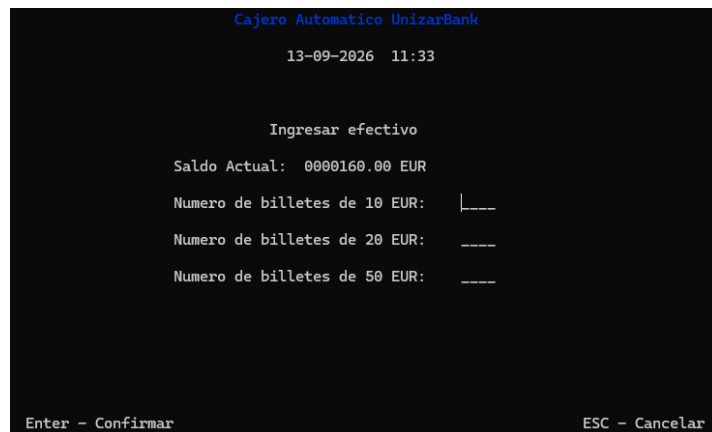


Figura 5 - Uso de billetes en “4- Ingresar efectivo”

Una vez confirmado el ingreso, el programa registra el movimiento en `movimientos.ubd`, actualiza el saldo de la cuenta y muestra una pantalla indicando tanto la cantidad ingresada como el nuevo saldo disponible. La funcionalidad se verificó realizando un ingreso con distintos billetes y comprobando posteriormente que el saldo y el historial de movimientos se actualizaban correctamente.

3.2. Objetivo 3b

A partir del análisis de los pantallazos y del código recuperado, se detectaron dos deficiencias notables de ergonomía visual en los esquemas de color:

1. Mensajes de error ilegibles: En las rutinas de excepción de todos los módulos (**PSYS-ERR**, **PINT-ERR**, **PPIN-ERR**, etc.), el texto de los avisos se presentaba con letras negras sobre fondo rojo (**BACKGROUND-COLOR IS BLACK BACKGROUND-COLOR IS RED**), lo cual ofrecía un contraste deficiente y de difícil lectura en terminales sobre fondo oscuro.
2. Cabeceras apagadas: El título corporativo “*Cajero Automatico UnizarBank*” utilizaba el color azul oscuro estándar (**BACKGROUND-COLOR IS 1 o BLUE**), que sobre el fondo negro de la pantalla presentaba muy baja visibilidad.

Para corregirlo, se realizó una refactorización de los atributos de color en la **PROCEDURE DIVISION** y **SCREEN SECTION** de todos los módulos (**BANK1.cbl** a **BANK9.cbl**):

- Se sustituyó el color del texto de los avisos de error por blanco sobre fondo rojo (**FOREGROUND-COLOR IS WHITE BACKGROUND-COLOR IS RED**), garantizando un contraste nítido e inmediato para el usuario.
- Se actualizaron los títulos y textos azules al color celeste brillante (**FOREGROUND-COLOR IS CYAN**), conservando la identidad corporativa de la entidad pero mejorando drásticamente la legibilidad general de la interfaz.

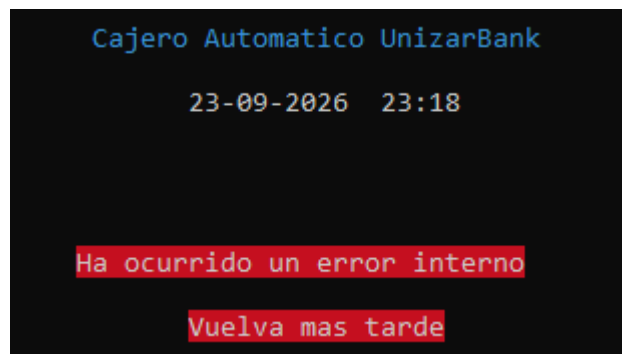


Figura 6 - Pantalla de aviso de error mostrando el nuevo contraste con texto blanco sobre fondo rojo y cabecera en azul celeste

3.3. Objetivo 3c

En diversas pantallas de la aplicación recuperada aparecía de forma persistente un dígito **0** (o la secuencia **0000**) en la esquina inferior derecha de la consola (fila 24, columna 80), careciendo de utilidad informativa para el usuario.

Causa técnica del problema:

Al analizar el código fuente se identificó que las pausas de confirmación y esperas de teclado (por ejemplo, tras mostrar el saldo o un aviso de error) se implementaban mediante sentencias como **ACCEPT (24, 80) PRESSED-KEY** o **ACCEPT CHOICE AT 2480**. Dado que en la **WORKING-STORAGE SECTION** dichas variables estaban declaradas con tipo numérico (**PIC 9(4)** o **PIC 9(1)**), el entorno de ejecución de COBOL inicializaba y proyectaba automáticamente en pantalla el contenido del campo rellenado con ceros por la izquierda en la coordenada de captura indicada.

Solución aplicada:

Para suprimir estos caracteres residuales sin alterar el control de flujo se aplicaron dos medidas:

1. Tipado alfanumérico: En las rutinas de pausa y espera de todos los submódulos (**BANK2.cbl** a **BANK9.cbl**), se modificó la variable de captura para que fuese de tipo carácter alfanumérico (**77 PRESSED-KEY PIC X(1).**). Al no ser un campo numérico, el compilador no dibuja ceros por defecto en la posición (24, 80). La captura de teclas de función (**Enter**, **ESC**, etc.) se mantiene intacta, ya que GnuCOBOL la gestiona de forma transparente mediante la directiva **CRT STATUS IS KEYBOARD-STATUS**.
2. Uso de la cláusula **BLANK ZERO**: En el menú principal (**BANK1.cbl**), donde se requiere capturar la opción numérica elegida, se canalizó la entrada a través de un elemento en la **SCREEN SECTION** dotado de la cláusula **BLANK ZERO (PIC 9(1) USING CHOICE)**. De este modo, mientras no se introduzca una cifra, la pantalla permanece completamente limpia.

3.4. Objetivo 3d

El diseño de la aplicación original de UnizarBank fue proyectado bajo el estándar clásico de terminales bancarios de texto y consolas DOS/CICS, cuya cuadrícula de visualización corresponde estrictamente a 80 columnas de ancho por 25 filas de alto. En las versiones modernas de Windows, la consola de comandos (**cmd.exe**) o Windows Terminal se abre por defecto con geometrías variables y dinámicas (generalmente de 120 columnas), lo que provocaba que las cabeceras, menús y tablas perdieran su centrado y proporción visual original.

Para asegurar que la aplicación se ejecute siempre de forma fija e invariable a 80x25 caracteres, se implementó un doble mecanismo de control:

1. Configuración en el script de arranque (**ejecutar.bat**): Se incorporó la orden del sistema **mode con: cols=80 lines=25** justo antes de invocar a **BANK1.exe**, garantizando que la ventana de la consola redimensione su búfer y cuadrícula visual de forma automática al hacer doble clic sobre el fichero.
2. Llamada nativa desde el código fuente (**BANK1.cbl**): Como medida de robustez adicional (por si el ejecutable es iniciado directamente por el usuario o por un evaluador sin hacer uso del script **.bat**), se añadió en la rutina **IMPRIMIR-CABECERA** del programa principal la instrucción COBOL: **CALL "SYSTEM" USING "mode con: cols=80 lines=25"**

De esta manera, el propio binario delega en la API del sistema operativo la fijación forzada de la geometría de 80 columnas y 25 líneas nada más arrancar la aplicación.

4. Empaquetado, distribución y control de versiones

4.1. Distribución standalone en Windows y dependencias de tiempo de ejecución

Con el fin de facilitar la corrección y permitir la ejecución inmediata de la aplicación haciendo doble clic sobre **BANK1.exe** en cualquier equipo con Windows sin exigir la instalación previa ni configuración del entorno GnuCOBOL, se investigó la posibilidad de compilar un binario 100% estático y monolítico.

Sin embargo, tras analizar el enlazador de GCC/MinGW y las cabeceras del compilador, se constató que no es técnicamente viable absorber la biblioteca **libcob** estáticamente dentro del ejecutable en Windows. La distribución binaria de MinGW/MSYS2 distribuye el runtime como biblioteca compartida (DLL); las funciones en C están decoradas con directivas **__declspec(dllimport)** y arrastran dependencias dinámicas obligatorias como Berkeley DB (**libdb**, necesaria para los ficheros indexados **.ubd**) y PDCurses (**pdcurses**, necesaria para la gestión de pantallas). Esto impide al enlazador (ld) resolver los símbolos de manera completamente estática sin recompilar todo el ecosistema desde el código fuente.

Por este motivo, se adoptó la siguiente solución de distribución:

1. Se unificaron todos los submódulos COBOL propios (**BANK1** a **BANK9**) dentro del ejecutable **BANK1.exe** mediante el flag de compilación estática interna **-fstatic-call**.
2. Se empaquetan en el mismo directorio de entrega las DLLs mínimas de tiempo de ejecución extraídas de GnuCOBOL (**libcob-4.dll**, **libdb-*.dll**, **pdcurses.dll**, etc.) junto a los ficheros de datos **.ubd**. De esta forma, el cargador de Windows resuelve las dependencias en la carpeta local, logrando una ejecución directa por doble clic completamente autónoma y funcional.

4.2. Repositorio de control de versiones (Git y GitHub)

Todo el ciclo de vida del proyecto, el historial de cambios, el código fuente, los programas auxiliares y la documentación técnica han sido gestionados mediante control de versiones en Git y se encuentran sincronizados y accesibles en el repositorio oficial de GitHub del grupo:

https://github.com/izancelorrio/Sistemas_legados.git

5. Reparto del trabajo

La práctica se ha desarrollado de forma colaborativa entre los tres miembros del grupo, distribuyendo las tareas principales por bloques funcionales para reducir dependencias y facilitar la integración posterior.

Alumno	Tareas	Horas aprox.
Alejo Beamonte Andreu (899577)	- Cumplimiento del Objetivo 1 y su respectiva redacción en la memoria. - Cumplimiento del Objetivo 3.a y su respectiva redacción en la memoria. - Redacción del tutorial de compilación y ejecución. - Desarrollo de los scripts de compilación y ejecución en Windows.	7 h
Izan Celorrio Caballero (900013)	- Cumplimiento del Objetivo 2 (ampliación de transferencias puntuales y periódicas en BANK6.cbl, creación de transferencias.ubd y módulo BANK9.cbl para el listado) y su redacción en la memoria. - Investigación sobre enlace estático, dependencias de libcob y despliegue standalone. - Gestión y sincronización del repositorio Git/GitHub.	7 h
Pablo Redondo Buenacasa (899595)	- Cumplimiento de los Objetivos 3.b, 3.c y 3.d (contraste de colores en errores y cabeceras, depuración y eliminación de ceros residuales en columna 80, forzado de la ventana a 80x25) y su redacción en la memoria. - Batería de pruebas cruzadas de las 8 opciones del cajero. - Revisión y maquetación final de la memoria técnica.	7 h